

Thrashing

서민재

오늘의 내용은?

01 개념

02 원인 및 흐름

03 대안 1 - 작업 집합

04 대안 2 - PFF

들어가기 전에

Page

가상 메모리를 사용하는 최소 크기 단위

Frame

물리 메모리를 사용하는 최소 크기 단위

Page-Fault

가상 메모리 주소를 통해 실제 물리 주소를 얻어 메모리에 접근했지만,
내가 원하는 페이지가 실제 메모리에 적재되어 있지 않을 때 발생

지역성

프로세스가 기억 장치의 정보를 시간적, 공간적으로 특정 부분을 집중적으로 참조

작업 집합

지역성에 기반하여 프로세스가 일정 시간 동안 원활하게 수행되기 위해
메모리에 올라와 있어야 하는 Page 집합

Thrashing (스래싱)



프로세스에 충분한 프레임이 없는 경우 → Page-Fault 발생



Page-Fault가 발생하는 경우, 페이지 교체가 필요

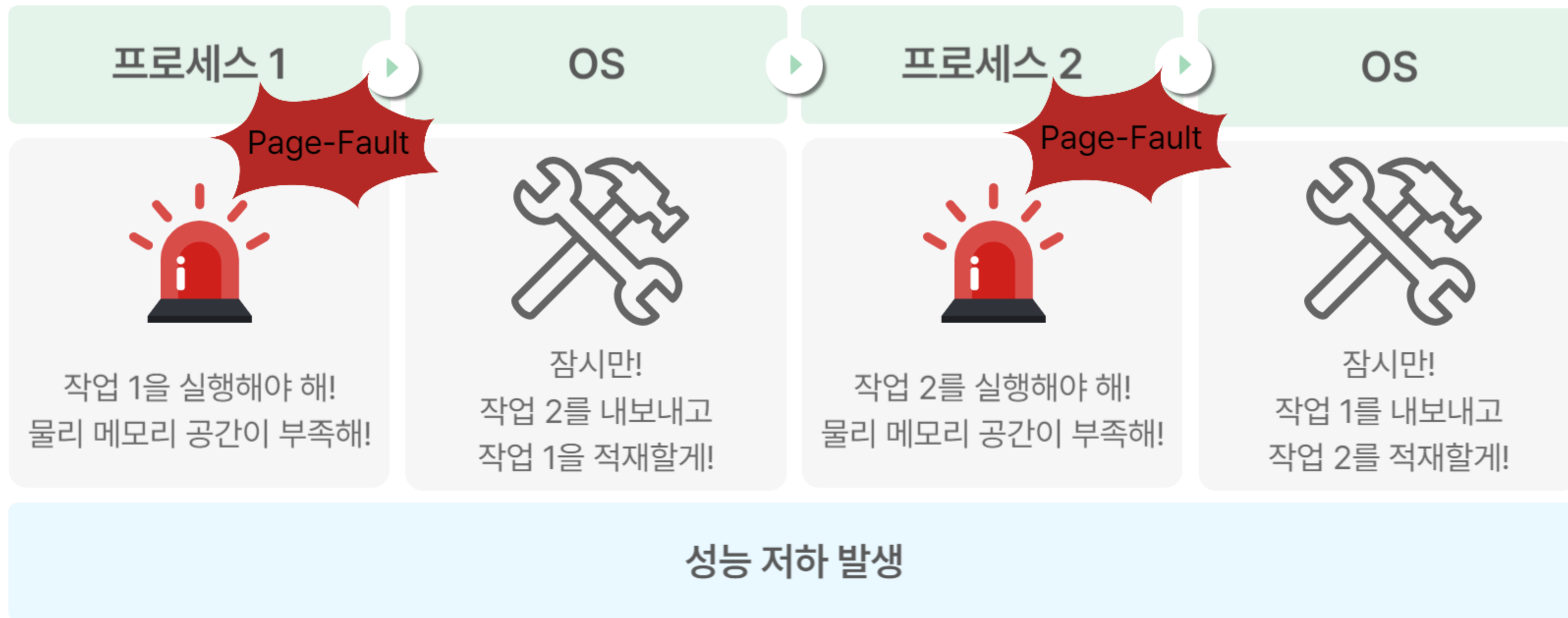


하지만, 이미 활발하게 사용되는 페이지들만으로 이루어져 있다면?



Page-Fault가 빈번하게 발생 → 실행보다 페이징에 더 많은 시간 발생

Thrashing (스래싱)



Page-Fault 발생

New Process

프로세스 A

프로세스 B



A0
A1
A2
A3
B1
B2
B3

프로세스가 새로 실행
↓
Page-Fault 발생
↓
다른 프로세스로부터
페이지 교체

A0
A1
A2
new0
B1
B2
B3

A3를 new0으로 교체
↓
프로세스A가 A3 접근
↓
Page-Fault 발생

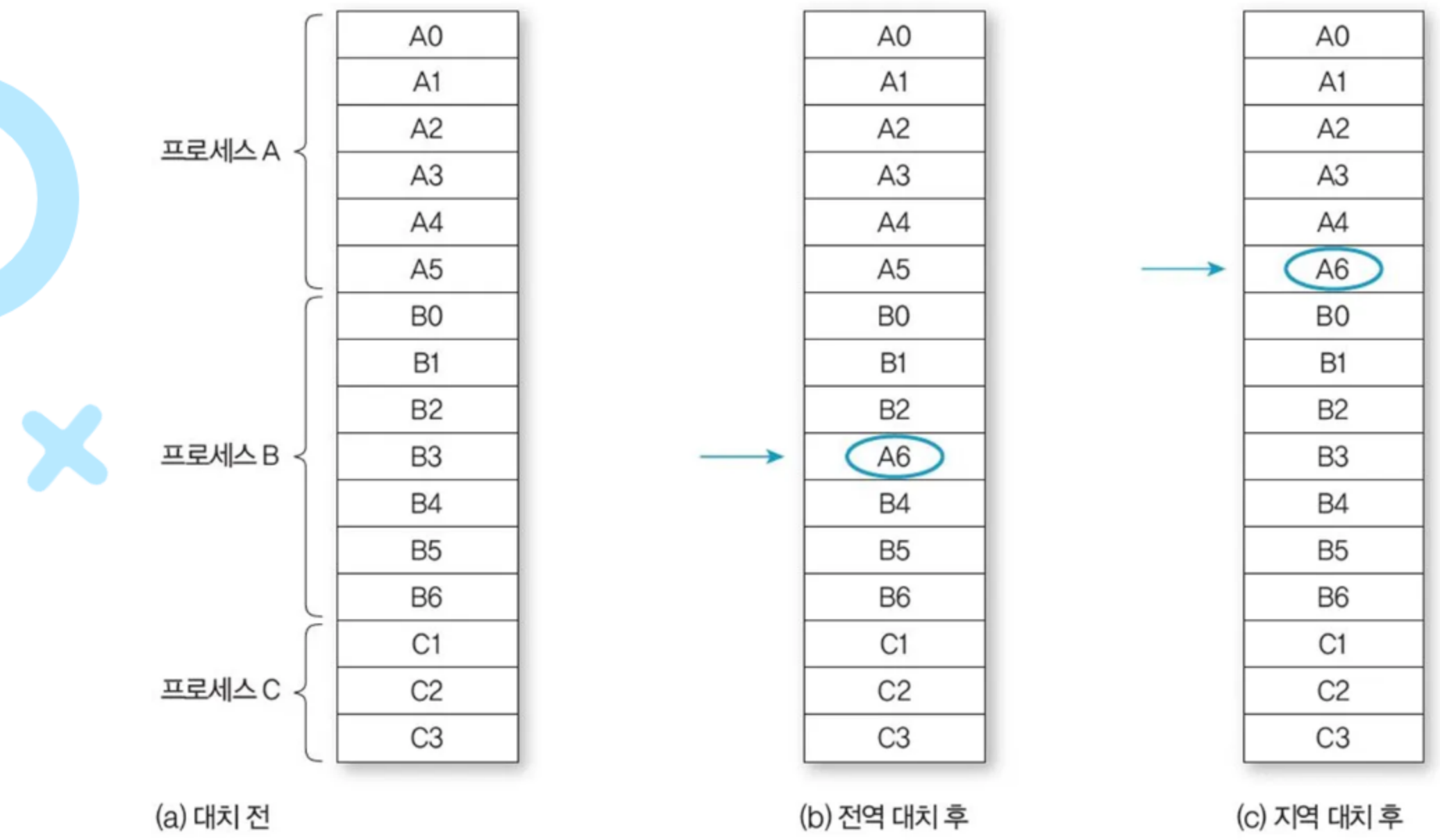
A0
A1
A2
new0
B1
A3
B3

B2를 A3로 교체
↓
프로세스B가 B2 접근
↓
Page-Fault 발생



*전역 페이지 교체 알고리즘: 모든 프로세스의 메모리 영역을 통틀어 교체 대상 페이지를 선정

흐름

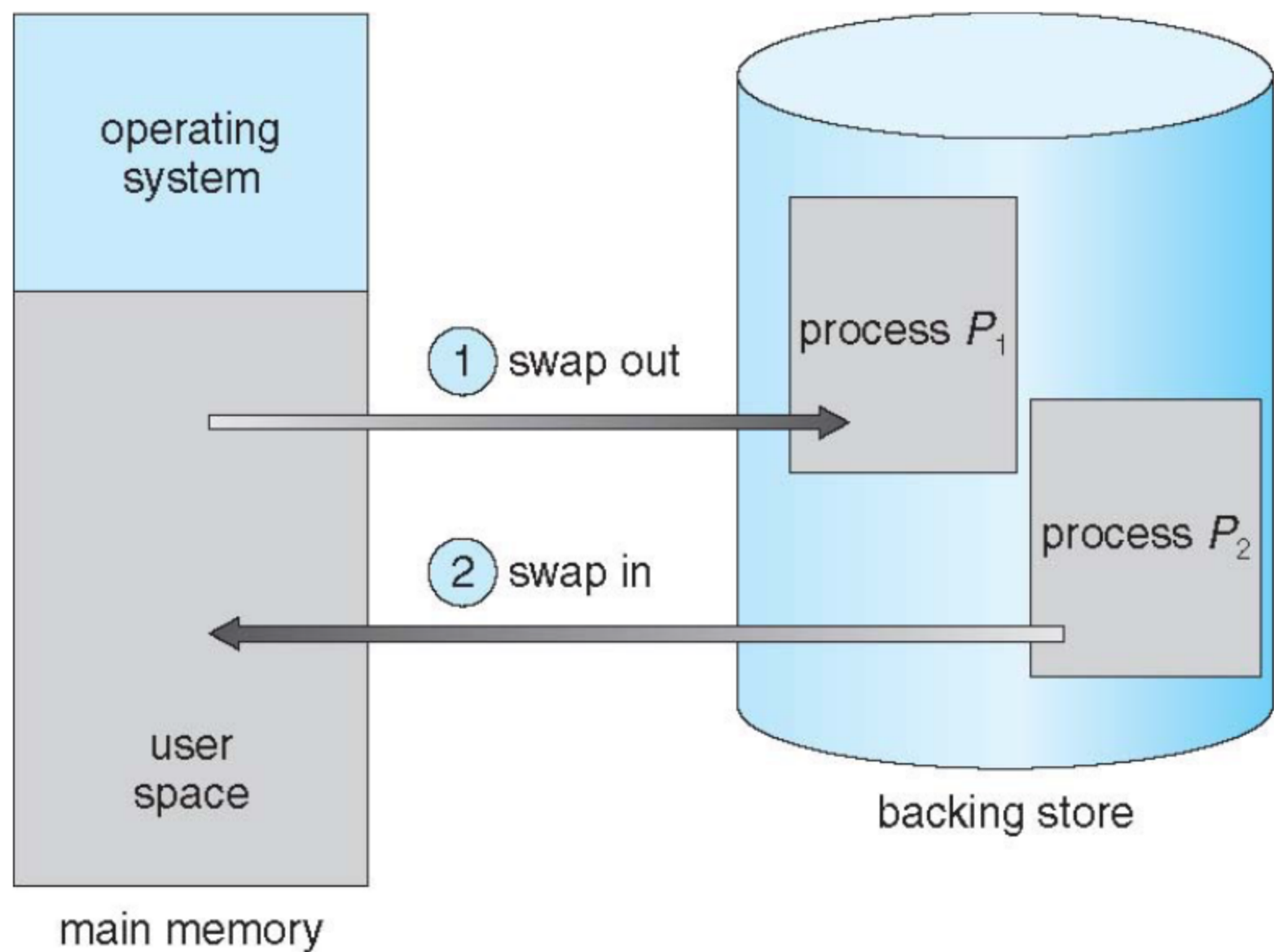


지역 페이지 교체 알고리즘
: 각 프로세스가 자신에게 할당된 프레임 세트에서만 선택
하나의 프로세스가 다른 프로세스의 프레임을 빼앗을 수 없다 → Thrashing 유발 불가능

그러면 전역 교체 알고리즘 말고 지역 교체 알고리즘 사용하면 되는 거 아니야?

그림 8-42 전역 대치와 지역 대치

Swapping



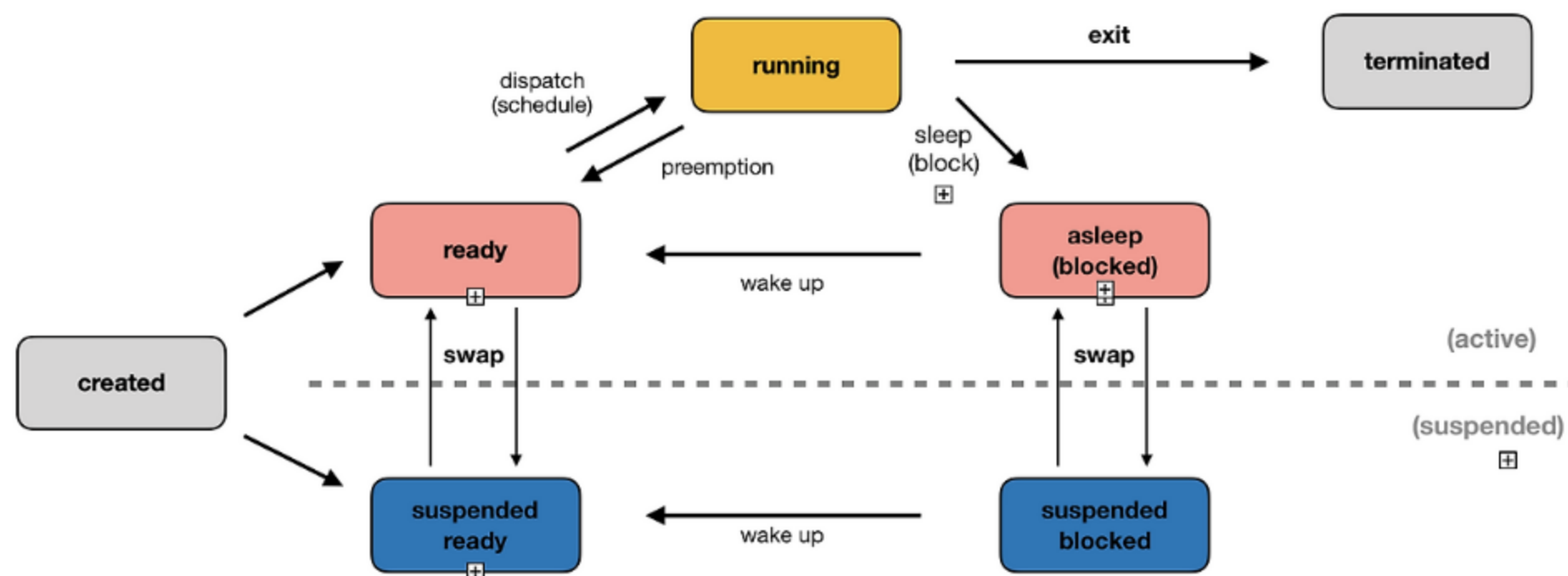
1. swap out

메인 메모리에서 페이지를 내보낼 때

2. swap in

메인 메모리에 페이지를 가져올 때

페이징



페이징 장치에 대한 큐잉 진행 → 준비 큐 empty

1. 프로세스 Page-Fault 발생
2. 디스크에서 페이지 읽기 시도
3. 페이징 장치 사용
4. 다른 프로세스가 사용 중으로 대기 큐 이동

페이징 장치 사용 전까지 CPU 사용 불가, Block 상태

⇒ 프로세스들이 페이징 장치를 기다리는 동안 CPU 이용률은 떨어진다.

Thrashing 발생

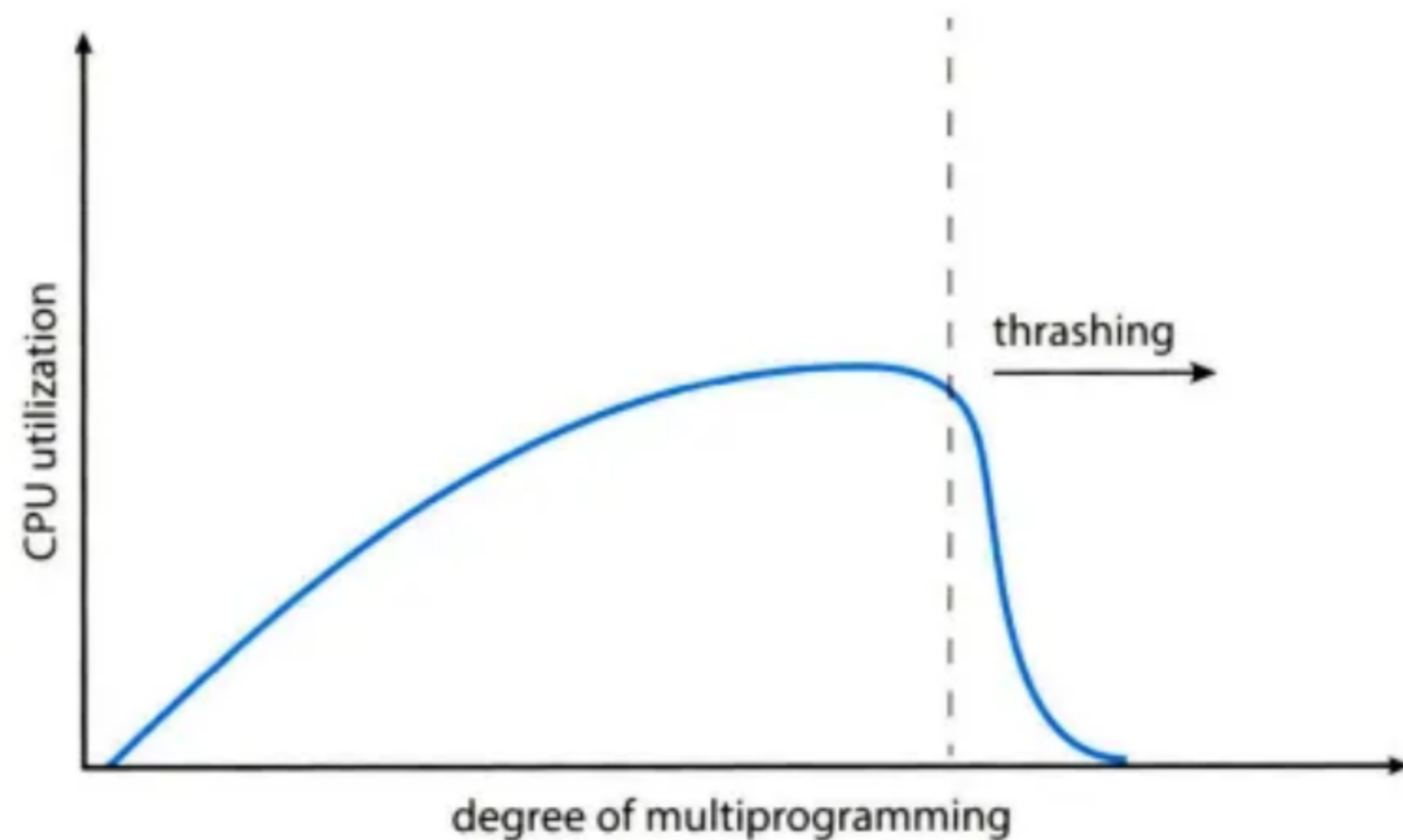


그림 10.20 스래싱(thrashing)

CPU 이용률 감소

- 새로운 프로세스 추가(다중 프로그래밍 정도 증가)
- 실행 중인 프로세스들로부터 프레임 가져옴
- Page-Fault와 긴 페이징 장치 대기 시간 발생

해당 지점에서는

CPU 이용률을 높이고 Thrashing을 중지하기 위해
다중 프로그래밍 정도를 낮춰야 한다

성능 저하



01. Thrashing 발생

Thrashing한 프로세스는 계속 Page-Fault
디스크에서 해당 페이지를 읽어와야 함
페이징 장치가 사용 중이면 대기 큐

02. 처리 시간 증가

대기열이 길어져
평균 Page-Fault 처리 시간 증가



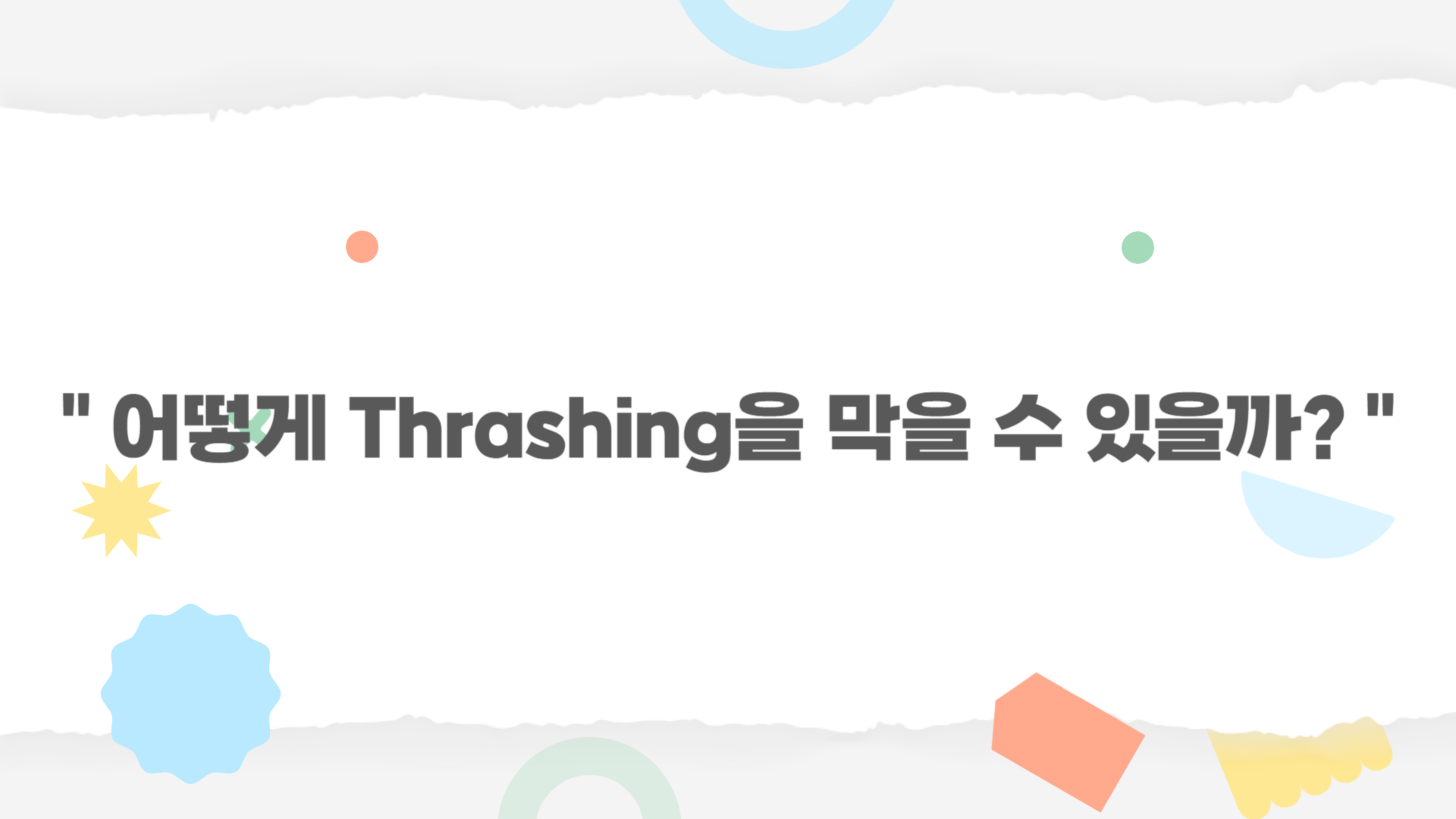
03. 여러 프로세스 영향

스래싱하지 않은 프로세스가 메모리 접근이 필요할 때
해당 프로세스도 데이터를 얻는 속도가 느려지고
CPU는 쉬게 됩니다.

04. 전체 성능 저하

전체 시스템의 실질 접근 시간 증가





" 어떻게 Thrashing을 막을 수 있을까? "

작업 집합 모델

page reference table

... 2 6 1 5 7 7 7 5 1 6 2 3 4 1 2 3 4 4 4 3 4 3 4 4 4 1 3 2 3 4 4 4 3 4 4 4 ...



그림 10.22 작업 집합 모델

- 시간적 지역성 기반
- window를 가지고 있다
- 일정 크기만큼의 페이지 참조를 관찰
- 한 프로세스가 최근 Δ (윈도우 크기)번 페이지를 참조했다면 그 안에 들어있는 서로 다른 페이지들의 집합
→ 작업 집합

- $\Delta = 10$
 - $WS(t_1) = \{1, 2, 5, 6, 7\}$
 - $WS(t_2) = \{3, 4\}$
- 작업 집합의 정확도는 Δ 의 선택에 따라 좌우
 - 너무 작을 경우, 지역의 전체를 포함하지 못함
 - 너무 클 경우, 여러 지역성을 과하게 수용

⇒ **집합의 크기**가 가장 중요하다

작업 집합 모델

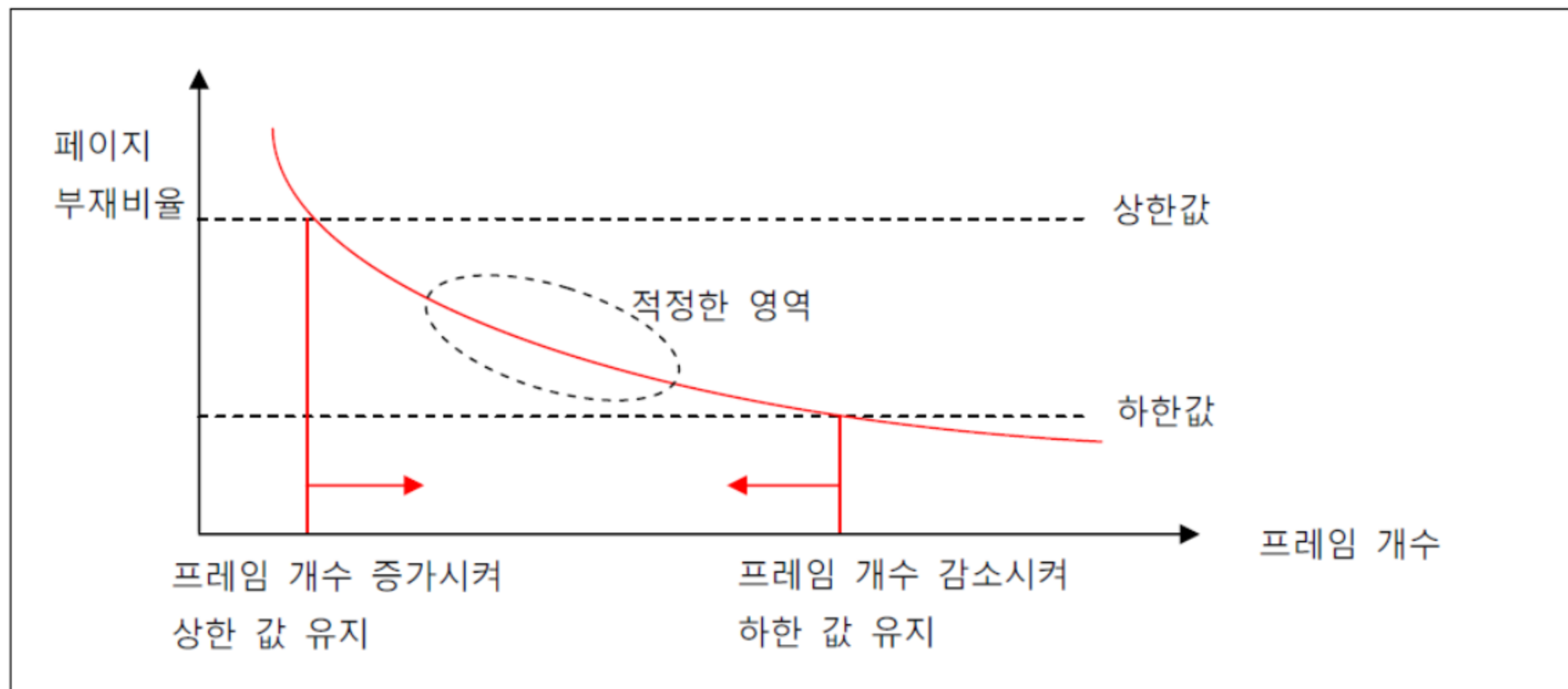
작업 집합을 어떻게 추적하지?

- 메모리 참조마다 한쪽에서는 새로운 페이지들이 추가되고 다른 쪽에서는 가장 오래된 참조부터 제외
→ 매 메모리 접근마다 작업 집합을 갱신해야 함

시간: t1 t2 t3 t4 t5 t6 t7
페이지: 1 2 3 2 4 5 6

- t5 시점 작업 집합 = {1, 2, 3, 4}
- t7 시점 작업 집합 = {2, 4, 5, 6}
- 작업 집합 window 내 어디에서라도 참조되는 페이지는 작업 집합에 포함 → 매번 작업 집합을 정확하게 추적
→ 오버헤드 발생

PFF (Page-Fault Frequency)



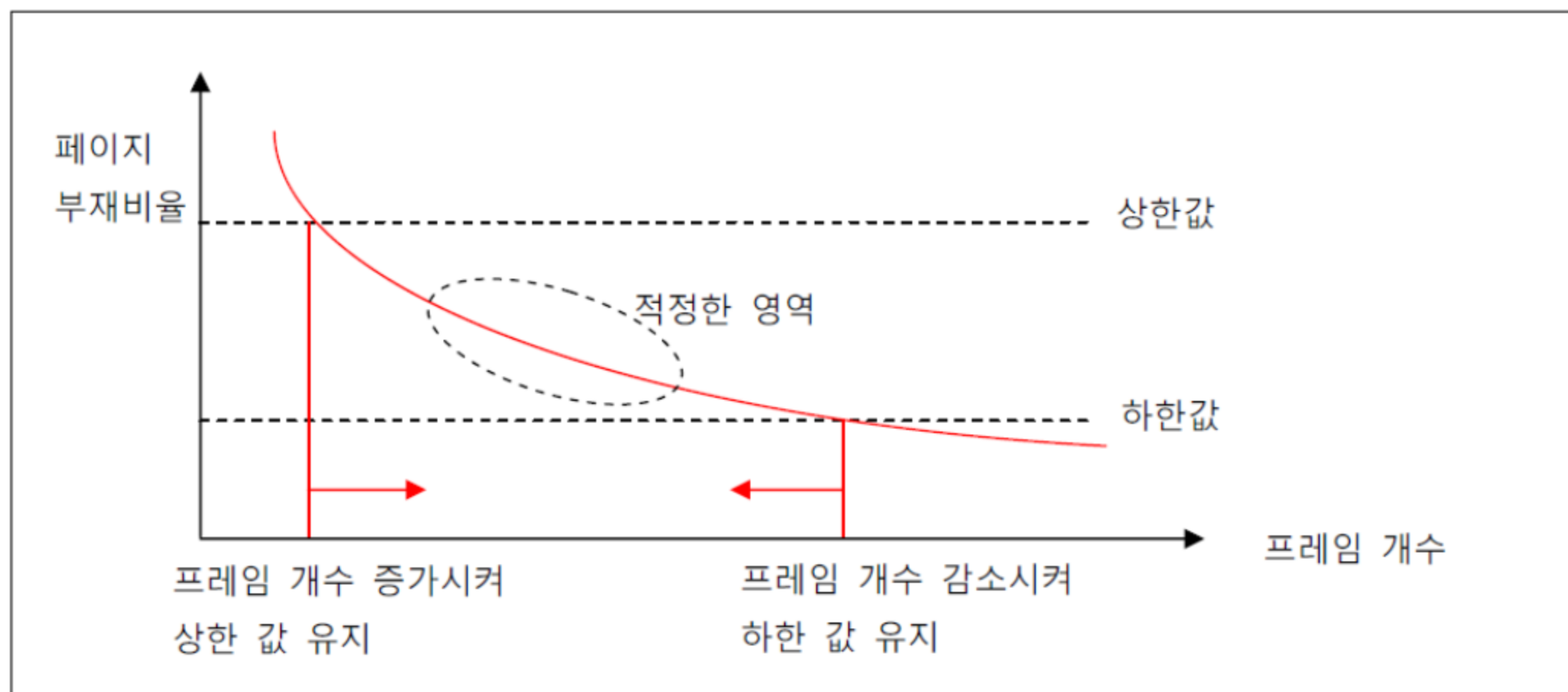
Page-Fault 비율이 높으면?

→ 더 많은 프레임이 필요하다

Page-Fault 비율이 낮으면?

→ 너무 많은 프레임을 갖고 있다

PFF (Page-Fault Frequency)



Page-Fault 비율이 높으면?

→ 상한 설정
→ 상한을 넘으면 프레임 추가 할당

Page-Fault 비율이 낮으면?

→ 하한 설정
→ 하한을 넘으면 프레임 줄이기

	기준	정확도	구현 난이도
WS (Working Set)	Δ (윈도우 크기) 동안 참조된 페이지 (작업 집합)	높음	높음
PFF (Page-Fault Frequency)	Page-Fault 발생 빈도	상대적 낮음 (근사 추정)	낮음

WS → 프로세스별 최소 작업 집합 크기 보장
PFF → 페이지 폴트를 기반 동적 조정

**⇒ 프로세스별로 최소 작업 집합은 반드시 확보하면서,
PFF에 따라 늘이거나 줄여 효율을 극대화하는 구조**



Thank you